

Ataques de envenenamiento de conocimiento a sistemas KG-RAG

Reproducción del pipeline de ataque en caja negra de Zhao et al. (arXiv:2507.08862)

Ingrid Alicia Yábar Geldres

Universidad Nacional de Ingeniería (UNI)
Facultad de Ingeniería Económica, Estadística y Ciencias Sociales
Doctorado en Ciencias e Ingeniería Estadística
ORCID: 0009-0003-9729-3259 ingrid.yabar.g@uni.pe

Contenido

- 1 Motivación
- 2 Marco teórico
- 3 Método de ataque implementado
- 4 Réplica del experimento
- 5 Desviaciones y limitaciones
- 6 Conclusiones

El artículo no publica código

Zhao et al. (2025) no incluye un repositorio de código ni un enlace a implementación oficial —verificado en el PDF/HTML del artículo y en su página de arXiv.

- No es posible clonar y ejecutar el código del artículo, como sí lo es para otros trabajos de la lista de este curso.
- Por eso, **todo lo que sigue es una implementación propia** del pipeline de tres etapas, construida directamente a partir de la descripción metodológica y las fórmulas del artículo (Ec. 1–7), no a partir de un código fuente existente.
- Las desviaciones frente al método original quedan documentadas de forma explícita en su propia sección, para que la implementación propia sea auditable frente al texto fuente.

¿Por qué importa la seguridad de KG-RAG?

- RAG permite que un LLM responda apoyándose en una fuente de conocimiento externa, en lugar de depender solo de lo memorizado en el entrenamiento.
- Cuando esa fuente es un **grafo de conocimiento (KG)**, el sistema puede seguir caminos de razonamiento explícitos y auditables entre entidades: KG-RAG.
- La misma propiedad que hace útil a un KG-RAG — un grafo editable y actualizable — es también su punto débil.
- Una sola tripleta falsa, bien conectada a la entidad correcta, puede desviar un camino de razonamiento multi-salto completo, sin alterar visiblemente el resto del grafo.

Pregunta

¿En qué país se filmó la película *Manchester By The Sea*?

Antes del ataque

Manchester By The Sea → filmPlace →
Manchester
Manchester → locatedIn →
Massachusetts
Massachusetts → stateOf → United
States

Después de insertar 1 arista falsa

Manchester → cityOf → England
England → containedIn → United
Kingdom
⇒ el sistema puede responder *United
Kingdom*

Objetivo de esta presentación: documentar una implementación propia de este ataque, de extremo a extremo, sobre un caso verificable a mano.

Grafo de conocimiento y caminos de relaciones

Grafo de conocimiento

$G = (E, R, T)$, con $T \subseteq E \times R \times E$ el conjunto de tripletas (e_h, r, e_t) : una arista dirigida y etiquetada de e_h a e_t .

Camino de razonamiento y camino de relaciones

$p : e_0 \xrightarrow{r_1} e_1 \xrightarrow{r_2} \dots \xrightarrow{r_l} e_l \Rightarrow$ camino de relaciones asociado $w = (r_1, r_2, \dots, r_l)$ (se descartan las entidades intermedias, se conserva solo la secuencia de relaciones).

Anclaje (*grounding*)

Operación inversa: dado un camino de relaciones w y una entidad de inicio, encontrar las entidades reales del grafo alcanzadas al recorrer w .

Pipeline de un sistema KG-RAG

Etapa de recuperación

$$G_q = \text{Retriever}(q; G) \quad (1)$$

$G_q = (E_q, R_q, T_q)$: subgrafo extraído del grafo completo G con solo la información relevante para responder la pregunta q .

Etapa de generación

$$a = f_{\theta}(\text{Prompt}[q, G_q]) \quad (2)$$

θ : parámetros del LLM generador; $\text{Prompt}[q, G_q]$: plantilla que combina la pregunta con el subgrafo recuperado; $a \in E$: respuesta final que produce el sistema.

Vulnerabilidad

Si G contiene tripletas falsas bien conectadas, la recuperación puede incluirlas en G_q sin distinguirlas de las legítimas.

Modelo de amenaza (caja negra)

$$\hat{A} = \text{KG-RAG}(q; \hat{G}), \quad a^* \notin \hat{A}, \quad \hat{G} = \{E, R, T \cup \hat{T}\} \quad (3)$$

- $\hat{T}$: tripletas adversarias insertadas; $\hat{G}$: grafo envenenado ($T \cup \hat{T}$); $\hat{A}$: respuestas que produce el sistema al operar sobre $\hat{G}$; a^* : respuesta correcta original.
- El atacante **no** conoce el recuperador, el LLM, ni la respuesta correcta a^* de la pregunta atacada.
- Su única capacidad es **insertar** $\hat{T}$ en el grafo — no puede borrar ni modificar tripletas existentes.
- Restricciones de sigilo: cada tripleta insertada reutiliza entidades y relaciones *ya existentes* en el grafo, y el número de tripletas por respuesta adversaria está acotado por un presupuesto K .

Formalización: objetivo de extracción de caminos (I–II)

$$\max_{\theta} \mathbb{E}_{w \sim W^*} [\log P_{\theta}(w \mid q)] \quad (4)$$

- θ : parámetros del modelo que propone caminos de relaciones $w = (r_1, \dots, r_l)$; W^* : caminos más cortos entre la entidad tema y la respuesta correcta — la supervisión débil del problema.
- Asumiendo w uniforme en W^* , la Ec. 4 se aproxima promediando el log-verosímil sobre W^* (Ec. 5, versión computable de la Ec. 4):

$$\arg \max_{\theta} \frac{1}{|W^*|} \sum_{w \in W^*} \log P_{\theta}(w \mid q) \quad (5)$$

$$\frac{1}{|W^*|} \sum_{w \in W^*} \log \prod_{i=1}^{|w|} P_{\theta}(r_i \mid r_{<i}, q) \quad (6)$$

- Un camino es una secuencia, así que su probabilidad se factoriza (Ec. 6): entrenar el modelo equivale a predecir, relación por relación, la siguiente más plausible dada la pregunta.
- El artículo resuelve esto **entrenando** un LLM específico de KG (LLM_RoG). Esta implementación logra el mismo efecto sin entrenamiento (ver sección de desviaciones).

Formalización: tripleta de perturbación

$$E_{l-1} = \text{Grounding}(e_q, w'; G), \quad w' = (r_1, \dots, r_{l-1}) \quad (7)$$

- e_q : entidad tema de la pregunta; $w' = (r_1, \dots, r_{l-1})$: **prefijo** del camino de relaciones w (todas sus relaciones menos la última, r_l); E_{l-1} : entidades reales alcanzadas al anclar w' desde e_q , recorriendo $e_q \xrightarrow{r_1} \dots \xrightarrow{r_{l-1}} e_{l-1}$.
- Por cada entidad alcanzada $e_{l-1} \in E_{l-1}$, se construye la tripleta de perturbación $(e_{l-1}, r_l, \hat{a})$, adjuntando la última relación del camino hacia la respuesta adversaria $\hat{a}$.
- **Estrategia de respaldo**: si el prefijo no se puede anclar, se sintetiza una cadena con entidades puente aleatorias que preserva el mismo patrón de relaciones hasta llegar a $\hat{a}$.

Pipeline de tres etapas

Etapa	Módulo	Salida
1. Respuestas adversarias	<code>adversarial_answers.py</code>	$\hat{A}$ (entidades del KG)
2. Caminos de relaciones	<code>relation_paths.py</code>	plantillas w
3. Inserción de perturbación	<code>perturbation.py</code>	tripletas nuevas

Orquestación

`run_attack(kg, question, topic_entity, llm, n_answers=5, budget_k=4)` encadena las tres etapas y devuelve el grafo envenenado.

Etapas 1 — Generación de respuestas adversarias

Question: {question}

Generate 5 entity names that are incorrect answers to this question, but might sound plausible or confusing.

- Only list the entity names as a bullet list.
- Each bullet should contain the name of one entity only.
- Do not include multiple distinct entities in a single bullet point.

- Prompt tomado de Zhao et al. (2025).
- El LLM puede alucinar nombres que no existen en el grafo.
- Se repiten rondas de generación y se aplica **coincidencia difusa** (`diffliib.get_close_matches`) contra el vocabulario de entidades del grafo hasta reunir N respuestas válidas.

Etapas 2 — Extracción de caminos de relaciones

- Se calcula el vocabulario de relaciones alcanzables en pocos saltos desde la entidad tema (`neighborhood_relations`).
- Se le pide al LLM caminos de relaciones usando **solo** ese vocabulario, con un prompt **propio de esta implementación** (el artículo resuelve esta etapa con un modelo entrenado, no con un prompt).
- Cualquier camino propuesto que use una relación fuera del vocabulario se descarta en el post-procesamiento.

Por qué funciona

Restringir el vocabulario garantiza caminos anclables en el grafo real, sin necesidad de entrenar un modelo específico de KG.

Etapas 3 — Inserción de tripletas de perturbación

Estrategia principal

- Ancla el prefijo del camino desde la entidad tema.
- Adjunta la última relación hacia la respuesta adversaria.
- Presupuesto K tripletas por respuesta adversaria.
- Nunca se duplica una tripleta ya existente en el grafo.

Estrategia de respaldo

- Si el prefijo no ancla, muestrea entidades puente al azar.
- Preserva el mismo patrón de relaciones del camino.

Entorno

```
pip install -r requirements.txt
cp .env.example .env # completar OPENAI_API_KEY

python scripts/demo_manchester.py --offline # referencia reproducible
python scripts/demo_manchester.py # API real, no determinista
```

- Grafo construido a mano: 13 tripletas sobre *Manchester By The Sea*.
- Pregunta atacada: ¿en qué país se filmó la película?
- Modo `-offline` (**referencia reproducible**): `StaticLLMClient` con respuestas fijas $\Rightarrow$ siempre inserta 2 tripletas hacia *United Kingdom*.
- Ejecución real contra `gpt-4o-mini` (temperatura 0.7, sin `-offline`): sin respuestas fijas, cada corrida puede generar candidatos distintos — ver siguiente diapositiva.

Ejecución con la API real de OpenAI

```
Adversarial target answers: ['Florida']
Relation path templates: [('filmPlace', 'locatedIn'), ('filmPlace', 'starring')]

Injected perturbation triples:
  (Manchester, locatedIn, Florida) [target: Florida]
  (Manchester, starring, Florida) [target: Florida]

Poisoned KG: 15 triples (+2 vs. clean)
```

- Modelo real (gpt-4o-mini, no determinista): sobrevivió la coincidencia difusa *Florida*, vía coincidencia accidental con (Miami, locatedIn, Florida), sin relación con la película.
- (filmPlace, starring) es **anclable pero semánticamente incoherente**: adjuntar starring a una ubicación no tiene sentido real, aunque la restricción de vocabulario lo admite.
- Con solo **2 tripletas insertadas** sobre 13 originales, se crean dos rutas hacia *Florida* sin ninguna señal estructural que las distinga de la ruta legítima hacia *United States* — confirma el mecanismo central del artículo, aun con un camino semánticamente incoherente.
- Límite honesto: sin semilla fija, esta corrida no es reproducible byte a byte.

Resultados reportados en el artículo original

Método (WebQSP)	F1 limpio	F1 atacado	Caída relativa
RoG	70.34	38.06	↓ 46 %
GCR	71.07	63.94	↓ 10 %
G-retriever	52.39	46.31	↓ 12 %
SubgraphRAG	66.94	50.22	↓ 18 %

- Con solo $N=5$ respuestas adversarias y $K=4$ tripletas cada una (máx. 20 tripletas por pregunta), los cuatro sistemas KG-RAG sufren caídas sustanciales frente a una línea base de modificación aleatoria (caída marginal).
- Hallazgo clave: la vulnerabilidad principal está en la etapa de **recuperación** (cobertura $>87\%$ de tripletas adversarias recuperadas); la etapa de generación es más resistente, pero exhibe una respuesta polarizada — o rechaza el contenido adversario por completo, o lo adopta predominantemente.

Extracción de caminos sin modelo entrenado

El artículo entrena LLM_RoG (LLaMA-2-7B) con el objetivo de las Ec. 4–6. Esta implementación logra caminos anclables restringiendo el vocabulario de relaciones de un LLM de propósito general al vecindario real de la entidad tema — sin GPU ni entrenamiento. El resto (respuestas adversarias por coincidencia difusa, inserción con estrategia de respaldo) sigue el artículo directamente.

- No se integra ningún sistema KG-RAG objetivo (RoG, GCR, G-retriever, SubgraphRAG): la validación se detiene en el grafo envenenado, sin medir métricas de respuesta final (Hit, F1, A-MRR, ...).
- Sin benchmarks a gran escala: un grafo de 13 tripletas construido a mano, frente a los miles de preguntas y subgrafos de WebQSP/CWQ.
- Modelo de lenguaje distinto: gpt-4o-mini en lugar de GPT-4 (detalle en la siguiente diapositiva).

Trabajo futuro: integrar un recuperador y generador mínimos para medir el efecto sobre la respuesta final; ejecutar sobre un subconjunto real de WebQSP/CWQ.

Modelos de lenguaje evaluados en el artículo

Categoría	LLM	F1 limpio	F1 atacado	Degradación
Open-source	Llama-2-7B-chat-hf	41.00	22.78	↓ 44 %
Open-source	Llama-3.1-8B-Instruct	54.20	32.02	↓ 41 %
Closed-source	GPT-3.5-turbo	56.51	36.19	↓ 36 %
Closed-source	GPT-4	61.33	37.31	↓ 39 %
MoE	DeepSeek-V3-0324	57.69	33.00	↓ 43 %
KG-specific	LLM_RoG (LLaMA-2-7B)	70.34	38.06	↓ 46 %

- Tabla 5 del artículo: robustez de RoG en WebQSP variando **solo** el LLM de generación, con recuperación y prompts fijos.
- Para las respuestas adversarias del ataque, el artículo usa **GPT-4**; para GCR/SubgraphRAG usa **GPT-3.5-turbo** como backend — ninguno de estos seis modelos coincide con el usado aquí.

Esta implementación

Usa únicamente gpt-4o-mini en las dos etapas basadas en LLM (respuestas adversarias y caminos de relaciones).

- Se reprodujo el pipeline de ataque de tres etapas de forma ejecutable y verificable, con una demostración de extremo a extremo contra la API real de OpenAI.
- El caso replicado confirma el mecanismo central del artículo: una perturbación mínima e indistinguible estructuralmente puede activar una cadena de inferencia engañosa completa.
- La principal simplificación — prescindir de un modelo entrenado específico de KG a favor de una restricción de vocabulario — preserva la propiedad esencial (camino anclable) sin requerir entrenamiento, pero no garantiza coherencia semántica: la ejecución real contra la API lo mostró con un camino anclable aunque semánticamente extraño.
- Próximo paso natural: cerrar la brecha hacia el artículo integrando un sistema KG-RAG objetivo para medir el impacto sobre la respuesta final.

Zhao, T., Chen, J., Ru, Y., Zhu, H., Hu, N., Liu, J., & Lin, Q. (2025). *RAG safety: Exploring knowledge poisoning attacks to retrieval-augmented generation*. arXiv.
<https://arxiv.org/abs/2507.08862>

¡Gracias!

Apéndice — Mapa de módulos: `src/kgrag_attack/`

Archivo	Qué hace
<code>kg.py</code>	Estructura <code>KnowledgeGraph</code> : alta de tripletas, vecindario de relaciones (<code>neighborhood_relations</code>), anclaje de caminos (<code>ground</code>), fusión grafo limpio + perturbación (<code>with_triples</code>).
<code>llm.py</code>	Protocolo <code>LLMClient</code> con dos implementaciones: <code>OpenAIClient</code> (API real, <code>gpt-4o-mini</code>) y <code>StaticLLMClient</code> (respuestas fijas, para pruebas y modo <code>-offline</code>).
<code>adversarial_answers.py</code>	Etapas 1. Pide al LLM respuestas incorrectas plausibles y las valida por coincidencia difusa contra las entidades reales del grafo.
<code>relation_paths.py</code>	Etapas 2. Pide al LLM caminos de relaciones restringidos al vocabulario real cercano a la entidad tema; descarta los que inventan relaciones.
<code>perturbation.py</code>	Etapas 3. Construye e inserta las tripletas de perturbación (estrategia principal de anclaje + respaldo con entidades puente), respetando el presupuesto K .
<code>attack.py</code>	Orquestador: <code>run_attack</code> encadena las tres etapas anteriores y arma el <code>AttackResult</code> con grafo limpio, grafo envenenado y todo lo generado.
<code>__init__.py</code>	Punto de entrada del paquete: expone <code>KnowledgeGraph</code> , <code>AttackResult</code> y <code>run_attack</code> para importarlos directamente desde <code>kgrag_attack</code> .

Apéndice — Secuencia de ejecución de `demo_manchester.py`

- ❶ `demo_manchester.py` agrega `src/` al `sys.path` e importa `run_attack`, `KnowledgeGraph` y los clientes LLM.
- ❷ Construye el grafo limpio con `KnowledgeGraph.from_triples(CLEAN_TRIPLES)`: 13 tripletas sobre *Manchester By The Sea* (`kg.py`).
- ❸ Instancia el LLM: `StaticLLMClient` con 2 respuestas fijas en modo `-offline`, o `OpenAIClient` (`gpt-4o-mini`) sin el flag (`llm.py`).
- ❹ Llama a `run_attack(kg, question, topic_entity, llm,...)` (`attack.py`), que ejecuta en orden:
 - ❶ `generate_adversarial_answers` (`adversarial_answers.py`)
 - ❷ `generate_relation_paths` (`relation_paths.py`)
 - ❸ `poison_knowledge_graph` (`perturbation.py`)
- ❺ Recibe el `AttackResult`: grafo limpio, grafo envenenado, respuestas adversarias, caminos de relaciones y tripletas insertadas.
- ❻ `demo_manchester.py` imprime el resumen y compara `out_edges` del grafo antes y después del ataque.

Para el expositor

Si preguntan "¿dónde vive cada pieza?": las 3 etapas del método están en `adversarial_answers.py`, `relation_paths.py` y `perturbation.py`; `kg.py` y `llm.py` son las utilidades compartidas que todas ellas usan; `attack.py` las encadena; `demo_manchester.py` es el único punto de entrada ejecutable desde la terminal.